RedCap Compression and Prism Automation Notes

Working formulation for block construction, field mapping, and Prism append automation

1) RedCap data compression into “equivalence classes” under bijection

Formally, this section defines a bijection set $\mathbf{B = \{\, z(x_i) : E(i, x) = S_z \,;\, i \in I \,\}}$ where the zth block $\mathbf{B_z}$ has an exact sequence pairing of events $\mathbf{S_z}$ that is unique to a specific set of instruments I, which are flattened into their unique fields $\mathbf{x_i}$ such that $\mathbf{z(x_j)}$ is the jth field in the zth column of a block matrix where all instruments are ordered as j. I is a set of x values, not a set of subsets of x values. That is, we do not distinguish between instruments if they have the same events, although in reality each instrument is a member set of I.

Intuition. An instrument is a data measuring device for select variables, where no other instrument can collect those same variables. Any event is an opportunity to use any number of devices to capture data, and any two events are an opportunity to compare data in time. By specifying a number of time points to sample, we are implicitly specifying events with select instruments.

RedCap in summary: events ordered in time, with instruments that capture fields unique to those instruments.

Given that we can extract all the data such that its structure lists the Subject ID as the first column and the event names as the second column, because instruments cannot intersect in fields but can intersect in events, we use the intersection of instruments along events to split the different fields into ordered groups in time.

Example instrument: *GridHamd* contains fields we may want to extract specifically, such as total score, though this can be specified with instructions. The PDF dictionary identifies its event sequence as:

```
GridHamd
visit_1_consenteva_arm_1
visit_13_cbtconclu_arm_3
visit_181monthfoll_arm_3
remote_2monthfollo_arm_3
remote_3monthfollo_arm_3
```

This data is identifying in two ways:

1. It is ordered in time, i.e. position along order.
2. It is labelled as a specific string.

Under this formulation scheme, the following sequence:

```
visit_1_consenteva_arm_1
visit_13_cbtconclu_arm_3
visit_181monthfoll_arm_3
remote_2monthfollo_arm_3
remote_3monthfollo_arm_3
```

is NOT the same as:

```
visit_1_consenteva_arm_1
visit_13_cbtconclu_arm_3
visit_181monthfoll_arm_3
remote_2monthfollo_arm_3
```

or:

```
visit_1_consenteva_arm_1
```

```
visit_13_cbtconclu_arm_3
remote_2monthfollo_arm_3
```

Therefore, by virtue of this distinction, whatever instrument we are looking at is not GridHamd. Further, each of these instruments is different from the others.

But consider exactly the same order in time and labels for the events:

```
gis Cgii And Gaf Prs
visit_1_consenteva_arm_1
visit_13_cbtconclu_arm_3
visit_181monthfoll_arm_3
remote_2monthfollo_arm_3
remote_3monthfollo_arm_3
```

We do not know which instrument this set of events belongs to. However, this takes care of all different cases based on time and label identity. This is enough to convert the global export into a well ordered set of blocks.

Canonical order: a fixed ordering relation.

Block, in this construction: any canonical ordering of events possible from instruments sharing a unique configuration in time, where they flatten irrecoverably from instruments into variables or fields.

In short: the grand export is converted into a block data structure where each block corresponds to a canonical ordering of events, and each canonical ordering of events is associated with one and only one set of instruments that have fields.

Although we have a well ordered set where the block now acts as an index over canonical events, we have no way of knowing what fields any block has just by knowing its order. To resolve this, we need a structure storing the information of fields × blocks, which is a reference for navigating the data structure created from the grand export.

This step does not require any assumptions because RedCap provides these structures directly.

2) .prism files as an appendable set of unwrapped JSONs

Prism's core is a collection of dictionaries that contain an ordering structure of data columns and graphing parameters. For our purposes, we are trying to automatically update data that is present.

Assumption 1, less important: *any graph is appended over only one variable, which is a single field from a possible set belonging to a particular instrument with a known canonical ordering.*

Assumption 2: *for any template that we have, at minimum we assume any graph is labelled reasonably, where "reasonably" means that given the Prism file string, we know which instrument or field it comes from, preferably both.*

This cannot really be checked automatically. Someone needs to verify that the PDF dictionary labels the variable we want to graph clearly enough that we can recover the correct Prism file, although this is a function of the name that we choose for Prism. Specifically, since the grand export contains the data columns in terms of the field variable name, which also has a readable label, and the Prism labelling is arbitrary, based on our choice, there needs to exist some mapping between the PDF label and the Prism file name we wish to append.

Summary of data flow:

```
Raw RedCap -> (instrument-event order + instrument-field variable) -> Well ordered dataset
```

```
(Field x Canonical Order index) -> Data source (needs Field variable name as input)
field-label + prism file name list -> field-prism link
```

But Prism needs Field Label as input, so use the field-label transform, i.e. the dictionary map.

With a field-prism link established, we can start appending. Notably, this is distinct from formatting, but it may be possible to change formatting as well depending on knowledge of the proper syntax. The point of the formulation up to this stage is to feed Copilot the metadata of all graphs of interest, so that it finds patterns across their JSON objects and creates a template mapping for each particular graph.

Entry test: start with cases where the mapping is clear within Prism: **1 Data table -> 1 graph** with no ambiguity. The metadata structure should make it clear what goes where and how after manual edits are made as desired in Prism itself.

3) Enough Formalism: how the script works

3. Download the .prism files you would like to update into a folder called PRISM in the script directory.
4. Read the metadata or use AI to create an executable template.
5. Adjust the template format based on Prism properties and desired design changes.
    - First CSV: placeholders for Sub##{graphname}, generalizing participant values and blanks, where non-empty values are always ordered canonically. There are n of these, where n is the number of graphs on deck.
    - Second CSV: sets of 3 columns for n graphs, i.e. 3n total, under graph_Ins[123]Sub##. Column 1 is for point color, column 2 is for line color across the series, and column 3 is for a specific Sub point alignment method embedded in the script, dependent on graph layout.
6. Set two folders:
    - PRISM: contains the graphs to update alongside the CSV templates above.
    - corestructure: contains an index of blocks and fields across them, plus the processed RedCap file, which may be updated regularly by running a separate script.
7. Run the automation script once the two CSV templates are set, and confirm the data layout through the printed DATA_{date} folder for each graph.

Reference objects

Object	Role in workflow
Canonical order	Fixed ordering relation over event labels and positions.
Block	Canonical event ordering shared by instruments with a unique time configuration after flattening into fields.
Field x block map	Reference structure for navigating processed block data.
Field-label	Dictionary map that connects RedCap field variables to readable labels needed for Prism matching.
Field-prism link	Mapping between a graph/template name and the field or label it should append.